

1. CASTING ENTRE OBJETOS (TYPE CASTING)

1.1 Idea clave

El casting permite **tratar un objeto como si fuera de otro tipo dentro de la misma jerarquía de herencia**.

Regla fundamental:

Solo se puede hacer casting entre clases que tengan relación de herencia.

1.2 Tipos de casting

1) Upcasting (conversión ascendente)

- De clase hija → clase padre
 - Es automático (no hace falta cast explícito)
 - Siempre es seguro
-

2) Downcasting (conversión descendente)

- De clase padre → clase hija
 - Necesita casting explícito
 - Puede fallar en ejecución
-

1.3 Ejemplo base

```
class Animal {  
  
    public void hacerSonido() {  
        System.out.println("Sonido genérico");  
    }  
}  
  
class Perro extends Animal {
```

```
public void ladrar() {  
    System.out.println("Guau");  
}  
}
```

1.4 Upcasting (automático)

```
public class Main {  
    public static void main(String[] args) {  
  
        Perro perro = new Perro();  
  
        Animal animal = perro; // upcasting automático  
  
        animal.hacerSonido(); // OK  
        // animal.ladrar(); ❌ No accesible  
    }  
}
```

Clave

- El objeto sigue siendo `Perro`
 - Pero la variable es `Animal`
 - Solo puedes usar lo que tenga `Animal`
-

1.5 Downcasting (explícito)

```
public class Main {  
    public static void main(String[] args) {  
  
        Animal animal = new Perro(); // upcasting  
  
        Perro perro = (Perro) animal; // downcasting  
  
        perro.ladrar(); // OK  
    }  
}
```

1.6 Error típico (muy importante)

```
Animal animal = new Animal();
```

Perro perro = (Perro) animal; // ✗ ERROR en ejecución

Esto lanza:

ClassCastException

1.7 Uso de instanceof (OBLIGATORIO en buenas prácticas)

```
if (animal instanceof Perro) {  
    Perro perro = (Perro) animal;  
    perro.ladrar();  
}
```

2. SOBRESCRITURA DE MÉTODOS (OVERRIDE)

2.1 Idea clave

Permite que una clase hija **modifique el comportamiento de un método heredado**.

2.2 Reglas

- Mismo nombre
 - Mismos parámetros
 - Mismo tipo de retorno
 - Usar `@Override` (muy recomendable)
-

2.3 Ejemplo claro

```
class Animal {  
  
    public void hacerSonido() {  
        System.out.println("Sonido genérico");  
    }  
}
```

```
}  
class Gato extends Animal {  
  
    @Override  
    public void hacerSonido() {  
        System.out.println("Miau");  
    }  
}
```

2.4 Ejecución

```
public class Main {  
    public static void main(String[] args) {  
  
        Animal a = new Gato();  
        a.hacerSonido();  
    }  
}
```

Salida:

Miau

2.5 Uso de super en override

```
class Perro extends Animal {  
  
    @Override  
    public void hacerSonido() {  
        super.hacerSonido();  
        System.out.println("Guau");  
    }  
}
```

Salida:

Sonido genérico
Guau

2.6 Ocultación de atributos

```
class Padre {  
    protected int valor = 10;  
}
```

```
class Hija extends Padre {
    protected int valor = 20;

    public void mostrar() {
        System.out.println(valor);    // 20
        System.out.println(super.valor); // 10
    }
}
```

3. POLIMORFISMO

3.1 Idea clave

Un mismo tipo (referencia) puede apuntar a objetos diferentes y comportarse distinto.

3.2 Ejemplo clásico

```
class Animal {

    public void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}

class Gato extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("Miau");
    }
}
```

3.3 Uso polimórfico

```
public class Main {  
    public static void main(String[] args) {  
  
        Animal[] animales = new Animal[3];  
  
        animales[0] = new Perro();  
        animales[1] = new Gato();  
        animales[2] = new Animal();  
  
        for (Animal a : animales) {  
            a.hacerSonido();  
        }  
    }  
}
```

Salida:

Guau
Miau
Sonido genérico

3.4 Clave fundamental

Se ejecuta el método de la clase real del objeto, no el de la referencia.

4. EJEMPLO COMPLETO (INTEGRANDO TODO)

```
class Empleado {  
  
    protected String nombre;  
    protected double sueldo;  
  
    public Empleado(String nombre, double sueldo) {  
        this.nombre = nombre;  
        this.sueldo = sueldo;  
    }  
  
    public void subirSueldo(double cantidad) {  
        sueldo += cantidad;  
    }  
}
```

```

    public double getSueldo() {
        return sueldo;
    }
}
class Ingeniero extends Empleado {

    private double incentivo;

    public Ingeniero(String nombre, double sueldo, double incentivo) {
        super(nombre, sueldo);
        this.incentivo = incentivo;
    }

    @Override
    public void subirSueldo(double cantidad) {
        sueldo += cantidad + incentivo;
    }

    @Override
    public double getSueldo() {
        return sueldo;
    }
}
class Directivo extends Empleado {

    public Directivo(String nombre, double sueldo) {
        super(nombre, sueldo);
    }

    @Override
    public void subirSueldo(double cantidad) {
        sueldo += cantidad * 1.2;
    }
}
public class Main {

    public static void main(String[] args) {

        Empleado[] empleados = new Empleado[3];

        empleados[0] = new Ingeniero("Carlos", 3000, 500);
        empleados[1] = new Directivo("Ana", 4000);
        empleados[2] = new Empleado("Luis", 2000);

        for (Empleado e : empleados) {
            e.subirSueldo(1000);
        }
    }
}

```

```
for (Empleado e : empleados) {  
    System.out.println(e.nombre + ": " + e.getSueldo());  
}  
}  
}
```

5. ERRORES COMUNES

- Hacer downcasting sin comprobar con `instanceof`
 - Pensar que el tipo de la variable determina el comportamiento (lo determina el objeto)
 - Confundir sobrecarga con sobrescritura
 - Intentar acceder a métodos de la subclase sin casting
-

6. RESUMEN FINAL

- Casting:
 - Upcasting: automático y seguro
 - Downcasting: explícito y peligroso
- Sobrescritura:
 - Cambia comportamiento heredado
 - Se usa `@Override`
- Polimorfismo:
 - Una referencia → múltiples comportamientos
 - Se ejecuta el método del objeto real

EJERCICIOS

1. Upcasting básico

Crea:

- `Animal` con método `hacerSonido()`
- `Perro` que sobrescriba ese método

En `main`:

- Crea un objeto `Perro`
- Asígnalo a una variable `Animal`
- Llama a `hacerSonido()`

Objetivo: comprobar qué método se ejecuta.

2. Downcasting seguro con `instanceof`

Crea:

- `Animal`
- `Gato` con método propio `maullar()`

En `main`:

- Crea un `Animal` que realmente sea un `Gato`
- Usa `instanceof` para convertirlo a `Gato`
- Llama a `maullar()`

3. Error de casting

Crea:

- `Animal`
- `Perro`
- `Gato`

En `main`:

- Crea un objeto `Animal` puro

- Intenta convertirlo a `Perro`
- Captura la excepción con try-catch

Objetivo: ver el `ClassCastException`.

4. Sobrescritura simple

Crea:

- `Vehiculo` con método `mover()`
- `Coche` que sobrescriba ese método

En `main`:

- Llama al método desde un objeto `Coche`
-

5. Uso de super en override

Amplía el ejercicio anterior:

- En `Coche.mover()`:
 - Llama primero a `super.mover()`
 - Luego añade comportamiento propio
-

6. Polimorfismo con array

Crea:

- `Animal`
- `Perro`
- `Gato`

En `main`:

- Crear un array `Animal[]`
- Guardar distintos tipos
- Recorrer y llamar a `hacerSonido()`

7. Método polimórfico

Crea:

- `Figura` con método `area()`
- `Rectangulo` y `Circulo`

Crea un método:

```
public static void imprimirArea(Figura f)
```

Dentro:

- Llamar a `area()`

En `main`:

- Pasar diferentes objetos

8. Casting + método específico

Crea:

- `Empleado`
- `Programador` con método `programar()`

En `main`:

- Guardar un `Programador` como `Empleado`
- Convertirlo a `Programador`
- Llamar a `programar()`

9. Sobrescritura en jerarquía real

Crea:

- `Empleado` con método `calcularSalario()`
- `Ingeniero` (añade incentivo)
- `Directivo` (incremento porcentual)

En `main`:

- Crear varios empleados
- Llamar al método desde referencia `Empleado`

10. Ejercicio integrador completo

Crea:

- `Animal` (método `hacerSonido()`)
- `Perro` (método `ladrar()`)
- `Gato` (método `maullar()`)

En `main`:

1. Crear lista `ArrayList<Animal>`
2. Añadir varios objetos
3. Recorrer:
 - Llamar a `hacerSonido()`
 - Si es `Perro`, llamar a `ladrar()`
 - Si es `Gato`, llamar a `maullar()`

Requisitos:

- Uso obligatorio de `instanceof`
- Uso de casting
- Uso de polimorfismo